

CLASIFICACIÓN DEL NIVEL DE HABILIDAD EN AJEDREZ MEDIANTE APRENDIZAJE AUTOMÁTICO



**UNIVERSIDAD
COMPLUTENSE
MADRID**

Asignatura: Aprendizaje Automático y Big Data

Grado: Ingeniería Informática

Autor: Sergio Vinagre Martínez

Fecha: 27 de Mayo de 2026

Índice de contenido

Índice de tablas	4
Índice de figuras	5
1. Introducción y objetivos	6
1.1 Contexto y motivación	6
1.2 Descripción del problema	6
1.3 Objetivos y alcance	7
2. Dataset	8
2.1 Descripción del conjunto de datos	8
2.2 Variable objetivo (target)	8
2.3 Muestreo estratificado y división de datos	9
2.4 Preprocesado, limpieza y preparación de variables	9
3. Análisis exploratorio de datos (EDA)	11
3.1 Distribución del ELO por nivel de habilidad	11
3.2 Distribución de clases	11
3.3 Movimientos por nivel	12
3.4 Forma de terminar la partida por nivel	13
3.5 Matriz de correlación	13
3.6 Control de tiempo por nivel	14
4. Preprocesado de datos	16
4.1 Feature engineering	16
4.2 Normalización	16
4.3 Balanceo de clases con SMOTE	16
5. Modelos de clasificación	18
5.1 Regresión logística One-vs-All (implementación NumPy)	18
5.2 Red neuronal feedforward (implementación NumPy)	19
5.3 Support vector machine con kernel RBF (scikit-learn)	21
5.4 Random Forest (scikit-learn)	22
5.5 Ensemble de votación suave (implementación NumPy)	23
6. Evaluación y comparación de modelos	24
6.1 Métricas de evaluación	24

6.2 Validación cruzada estratificada 5-Fold.....	24
6.3 Curvas ROC	24
6.4 Matrices de confusión	28
6.5 Resultados con threshold por defecto (0.5).....	31
6.6 Ajuste de threshold para mejorar el recall de intermedios	32
6.7 Análisis del comportamiento de generalización.....	33
6.8 Modelo recomendado	35
7. Manual de usuario	37
7.1 Requisitos del sistema y entorno	37
7.2 Estructura del código fuente	37
7.3 Instrucciones de ejecución	38
7.4 Artefactos y resultados del pipeline.....	38
7.5 Simulación del despliegue en producción	39
8. Conclusiones	40
8.1 Resumen de resultados	40
8.2 Limitaciones	40
8.3 Posibles mejoras	41
9. Bibliografía	42

Índice de tablas

<i>Tabla 1 - Principales columnas del dataset Lichess Games.</i>	8
<i>Tabla 2 - División train / validación / test con estratificación.</i>	9
<i>Tabla 3 - Features construidas y transformaciones aplicadas (total: 17 columnas).</i>	16
<i>Tabla 4 - Grid de hiperparámetros para Regresión Logística OvA.</i>	18
<i>Tabla 5 - Grid de hiperparámetros para la Red Neuronal.</i>	21
<i>Tabla 6 - Grid de hiperparámetros para SVM RBF.</i>	22
<i>Tabla 7 - Grid de hiperparámetros para Random Forest.</i>	22
<i>Tabla 8 - Resultados de los cinco modelos en el conjunto de prueba (threshold = 0.5).</i>	31
<i>Tabla 9 - Resultados con threshold ajustado para maximizar recall de Intermedios.</i>	32

Índice de figuras

<i>Figura 1 - Distribución del ELO promedio por clase con percentiles 33 y 66.</i>	11
<i>Figura 2 - Distribución de clases en la muestra de 2.500 partidas.</i>	12
<i>Figura 3 - Distribución de movimientos totales (turns) y de apertura (opening_ply) por nivel.</i>	12
<i>Figura 4 - Distribución de victory_status por nivel de habilidad.</i>	13
<i>Figura 5 - Matriz de correlación de Pearson entre features numéricas.</i>	14
<i>Figura 6 - Distribución del tiempo base del control de tiempo por nivel de habilidad.</i>	15
<i>Figura 7 - Curva de validación de lambda_reg para Regresión Logística (5-fold CV).</i>	19
<i>Figura 8 - Importancia de features según los coeficientes de la Regresión Logística (clase Intermedio).</i>	19
<i>Figura 9 - Evolución de la perdida y el accuracy por época en train y validación. La línea verde marca el punto de Early Stopping.</i>	20
<i>Figura 10 - Curva de validación del learning_rate para la Red Neuronal (5-fold CV).</i>	21
<i>Figura 11 - Importancia relativa de features según el Random Forest.</i>	23
<i>Figura 12 - Curva ROC - Regresión logística (AUC = 0.706).</i>	25
<i>Figura 13 - Curva ROC - Red Neuronal (AUC = 0.737).</i>	26
<i>Figura 14 - Curva ROC - SVM RBF (AUC = 0.646).</i>	26
<i>Figura 15 - Curva ROC - Random Forest (AUC = 0.711).</i>	27
<i>Figura 16 - Curva ROC - Ensemble Soft Voting (AUC = 0.726).</i>	27
<i>Figura 17 - Matriz de confusión - Regresión logística (threshold=0.5).</i>	29
<i>Figura 18 - Matriz de confusión - Red Neuronal (threshold=0.5).</i>	29
<i>Figura 19 - Matriz de confusión - SVM RBF (threshold=0.5).</i>	30
<i>Figura 20 - Matriz de confusión - Random Forest (threshold=0.5).</i>	30
<i>Figura 21 - Matriz de confusión - Ensemble Soft Voting (threshold=0.5).</i>	31
<i>Figura 22 - Matriz de confusión - Ensemble con threshold=0.410 (modelo recomendado).</i>	33
<i>Figura 23 - Curva de aprendizaje - SVM RBF.</i>	34
<i>Figura 24 - Curva de aprendizaje - Random Forest.</i>	35
<i>Figura 25 - Comparativa de modelos</i>	35

1. Introducción y objetivos

En los últimos años, el crecimiento del ajedrez online ha generado una cantidad masiva de datos procedentes de partidas disputadas diariamente en plataformas como Lichess. Estas partidas, además de reflejar decisiones estratégicas, dejan un rastro de variables medibles (tiempo, tipo de victoria, número de turnos, apertura, etc.) que permiten analizar el comportamiento de juego de forma cuantitativa. En este contexto, el aprendizaje automático ofrece herramientas para identificar patrones y construir modelos capaces de inferir información relevante a partir de características observables, sin necesidad de apoyarse directamente en indicadores explícitos como la puntuación ELO.

1.1 Contexto y motivación

El ajedrez se considera uno de los dominios clásicos para el estudio de la toma de decisiones y la estrategia y, en la actualidad, gran parte de su práctica se desarrolla en entornos digitales. Plataformas como Lichess almacenan cada partida junto con múltiples atributos descriptivos, lo que abre la puerta a realizar análisis de datos a gran escala y a diseñar sistemas inteligentes de apoyo a jugadores, entrenadores y organizadores.

En este proyecto se aborda un problema de clasificación supervisada orientado a estimar el nivel de habilidad del jugador (dos categorías: Principiante e Intermedio) utilizando únicamente información accesible en los metadatos de la partida, evitando emplear directamente la puntuación ELO como variable de entrada. La motivación principal es que, aunque el ELO sea un indicador sólido del nivel, no siempre está disponible o no siempre es deseable utilizarlo como “atajo”. En cambio, muchas señales indirectas de la habilidad pueden manifestarse en estadísticas de juego observables y comparables entre partidas.

Desde un punto de vista práctico, un enfoque de este tipo puede resultar útil para tareas como el análisis pedagógico del rendimiento, la personalización de recomendaciones de entrenamiento o la mejora de sistemas de emparejamiento, aportando una caracterización del nivel basada en el comportamiento reflejado en la partida.

1.2 Descripción del problema

El problema que se aborda en este trabajo consiste en inferir el nivel de habilidad de un jugador a partir de la información observable que deja una partida de ajedrez. En particular, se plantea como un problema de clasificación en el que, dados los datos descriptivos de una partida (por ejemplo, número de turnos, tipo de victoria, información de apertura y parámetros de tiempo), el sistema debe determinar a qué categoría de habilidad pertenece el jugador.

Aunque la calidad de juego suele expresarse mediante la puntuación ELO, este proyecto se centra en un enfoque más general: no utilizar el ELO como entrada del modelo, sino aprender a reconocer patrones presentes en las estadísticas de la partida. Para ello, el nivel de habilidad se formula como una etiqueta discreta que permite entrenar y evaluar modelos de clasificación.

En el conjunto de datos utilizado se definen inicialmente tres niveles (Principiante, Intermedio y Avanzado). No obstante, el alcance principal del proyecto se centra en la diferenciación entre Principiante e Intermedio, por ser un escenario especialmente relevante en entornos reales: es donde suele concentrarse un mayor volumen de jugadores y donde una detección temprana del progreso resulta más útil para personalizar el aprendizaje y la experiencia de juego.

1.3 Objetivos y alcance

El objetivo general es el de desarrollar y evaluar un sistema de clasificación capaz de predecir el nivel de habilidad (Principiante/Intermedio) a partir de características derivadas de partidas de ajedrez, utilizando técnicas de aprendizaje automático y evitando el uso directo del ELO como variable de entrada.

En cuanto a los objetivos específicos nos encontramos con los siguientes:

1. Analizar y preparar un conjunto de partidas de ajedrez, identificando variables relevantes y estableciendo un formato adecuado para su tratamiento.
2. Definir una representación de cada partida mediante un conjunto de características interpretables basadas en información disponible en los metadatos.
3. Entrenar y comparar distintos modelos de clasificación con el fin de estudiar su rendimiento y sus diferencias de comportamiento.
4. Evaluar los modelos mediante métricas apropiadas para clasificación, identificando fortalezas y limitaciones de cada enfoque.
5. Seleccionar y documentar la solución con mejor equilibrio entre rendimiento y aplicabilidad, dejando el proyecto preparado para su reproducción y uso posterior.

En cuanto al alcance del proyecto, este se centra en clasificación de nivel a partir de una partida utilizando variables derivadas de metadatos y estadísticas asociadas. El estudio principal se limita a dos categorías (Principiante e Intermedio), dejando el nivel Avanzado como referencia para futuras extensiones o como parte de un planteamiento multiclase. El resultado esperado es un sistema experimental reproducible que permita comparar enfoques y extraer conclusiones sobre la viabilidad de estimar el nivel sin depender de la puntuación ELO como entrada.

2. Dataset

Para el desarrollo del proyecto, se utilizará el conjunto de datos público "Chess Game Dataset" (autor: datasnaek), alojado en la plataforma Kaggle. Este dataset contiene registros de partidas reales jugadas en el servidor Lichess. El link es el siguiente:

<https://www.kaggle.com/datasets/datasnaek/chess>

2.1 Descripción del conjunto de datos

El conjunto de datos utilizado corresponde al Lichess Games Dataset, disponible públicamente en Kaggle. Contiene información de partidas disputadas en la plataforma Lichess e incluye variables relacionadas con el desarrollo y el resultado de la partida (por ejemplo, número de turnos, forma de victoria o control de tiempo), así como información asociada a la apertura. Adicionalmente, el dataset incorpora los ratings (ELO) de ambos jugadores, que en este proyecto se emplean únicamente como referencia para construir la etiqueta de habilidad, pero no como variables de entrada al modelo.

Las columnas principales del dataset original se resumen en la Tabla 1.

Columna	Descripción
id	Identificador único de la partida
rated	Si la partida conto para el ranking (booleano)
turns	Número total de movimientos de la partida
victory_status	Forma de terminar: mate, resign, outoftime, draw
winner	Ganador: white, black o draw
increment_code	Control de tiempo en formato "base+incremento"
white_rating	Puntuación ELO del jugador con blancas
black_rating	Puntuación ELO del jugador con negras
opening_eco	Código ECO de la apertura jugada
opening_ply	Numero de movimientos de apertura reconocidos

Tabla 1 - Principales columnas del dataset Lichess Games.

2.2 Variable objetivo (target)

El objetivo del proyecto es predecir el nivel de habilidad sin utilizar directamente el ELO como entrada. Sin embargo, al tratarse de un problema supervisado, es necesario definir una etiqueta que actúe como referencia de "ground truth". Dado que el dataset no proporciona directamente la categoría de habilidad, esta se construye a partir del ELO de los jugadores como una aproximación objetiva del nivel.

1. Se calcula el ELO promedio por partida: $\text{skilllevel} = (\text{White_rating} + \text{Black_rating}) / 2$.
2. Se calculan los percentiles 33 y 66 sobre el dataset completo (antes de aplicar muestreo).
3. Se discretiza skill_level en tres categorías: Principiante: valores por debajo del percentil 33, intermedio: valores entre percentil 33 y 66 y avanzado: valores por encima del percentil 66.

Este planteamiento permite disponer de una etiqueta consistente para el entrenamiento sin incorporar los ratings como variables predictoras. En consecuencia, durante la inferencia el modelo se basa exclusivamente en características observables de la partida.

2.3 Muestreo estratificado y división de datos

Para mantener tiempos de entrenamiento y experimentación razonables, se trabaja con una muestra estratificada de 2.500 partidas. El muestreo estratificado se utiliza para preservar, en la medida de lo posible, la proporción de clases definida en la variable objetivo, evitando que una clase quede infrarrepresentada por efecto del muestreo aleatorio.

Tras obtener la muestra, el conjunto se divide en subconjuntos de entrenamiento, validación y prueba, manteniendo la estratificación. La Tabla 2 muestra la distribución utilizada.

Conjunto	Tamaño	proporción
Train	1.500	60 %
Validacion	500	20 %
Test	500	20 %

Tabla 2 - División train / validación / test con estratificación.

2.4 Preprocesado, limpieza y preparación de variables

Antes de entrenar los modelos, se realiza un preprocesado con el objetivo de transformar el archivo original (games.csv), que contiene más de 20.000 partidas, en un conjunto de datos manejable, consistente y apto para clasificación. En primer lugar, para reducir el coste computacional y ajustarse al alcance del proyecto, se trabaja con una muestra estratificada de 2.500 partidas, preservando la distribución de clases derivada de la variable objetivo. A partir de esta muestra se aplican las siguientes tareas de preparación:

Selección y construcción de variables (feature engineering): se generan las variables de entrada a partir de metadatos de la partida, incluyendo información de apertura, duración (turns), forma de finalización, control de tiempo y otros atributos disponibles que describen el desarrollo de la partida.

Codificación de variables categóricas: las variables no numéricas (por ejemplo, tipo de finalización o atributos de apertura) se convierten a representaciones numéricas para poder ser procesadas por los modelos.

Procesado del control de tiempo: el campo increment_code se descompone para obtener variables numéricas interpretables (por ejemplo, tiempo base e incremento).

Escalado/normalización: las variables numéricas se ajustan a escalas comparables para facilitar el entrenamiento y mejorar la estabilidad del aprendizaje.

Prevención de fuga de información: los ratings (`white_rating` y `black_rating`) se utilizan únicamente para construir la etiqueta (Sección 2.2) y se excluyen del conjunto de variables de entrada, garantizando que el modelo no depende directamente del ELO.

Con este preprocesado se obtiene una matriz final de características lista para el entrenamiento, validación y evaluación comparativa de los modelos propuestos.

3. Análisis exploratorio de datos (EDA)

Antes del modelado se realizó un análisis exploratorio completo para comprender la distribución de las variables, comprender distribuciones, detectar desbalance y motivar decisiones de preprocesado. A continuación, se presentan las seis visualizaciones principales generadas.

3.1 Distribución del ELO por nivel de habilidad

El histograma muestra la distribución del ELO promedio separada por clase, con líneas verticales en los percentiles 33 y 66. Se observa que las tres clases se solapan en los extremos, lo que anticipa la dificultad de separación que mostraran los modelos. El solapamiento entre Principiante e Intermedio es el que más afecta al rendimiento, ya que son las dos clases que el sistema debe distinguir.

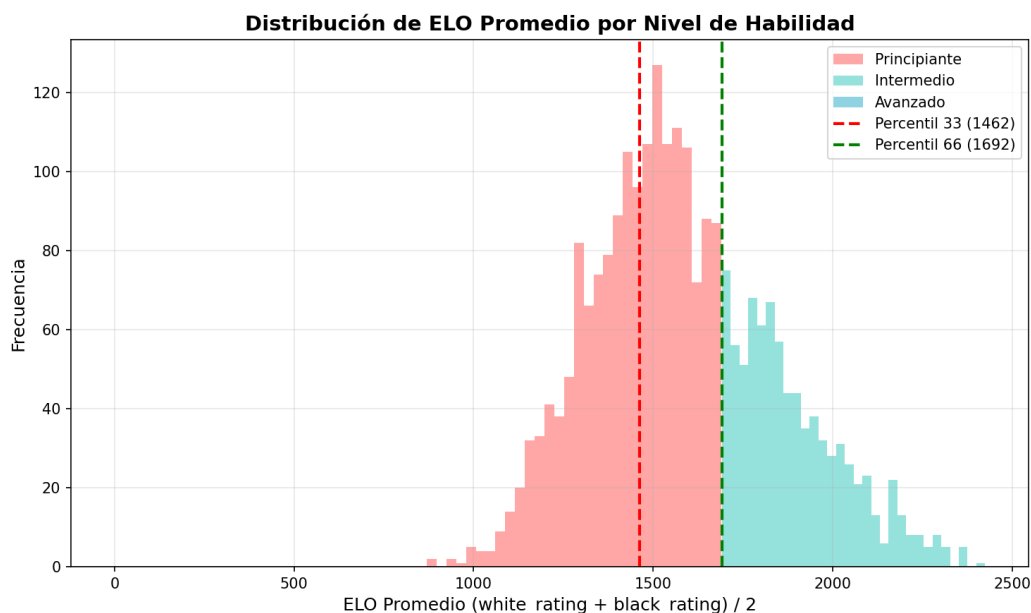


Figura 1 - Distribución del ELO promedio por clase con percentiles 33 y 66.

3.2 Distribución de clases

El gráfico de barras confirma que el muestreo estratificado preserva, en la muestra de 2.500 partidas, una distribución de clases coherente con la del conjunto original. No obstante, al centrar el estudio en el escenario binario (Principiante vs. Intermedio), la distribución resultante presenta un desbalance aproximado del 66% / 34%. Este desequilibrio puede sesgar el entrenamiento hacia la clase mayoritaria, por lo que se justifica el uso de técnicas de balanceo (como SMOTE, aplicado únicamente en entrenamiento) y el empleo de métricas menos sensibles al desbalance, como F1 y ROC-AUC.

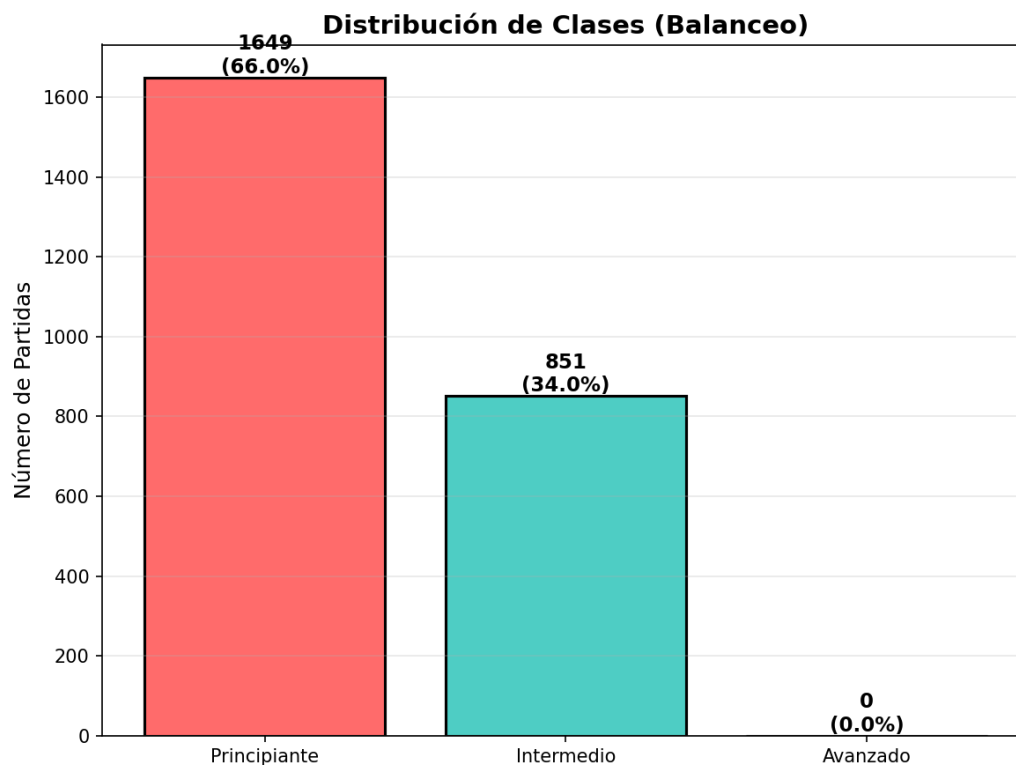


Figura 2 - Distribución de clases en la muestra de 2.500 partidas.

3.3 Movimientos por nivel

Los boxplots de turns y opening_ply por clase muestran que los jugadores de nivel superior tienden a jugar partidas con más movimientos y utilizan aperturas más elaboradas. La varianza es alta en todas las clases, indicando que estas variables por si solas no son suficientes para separar las clases.

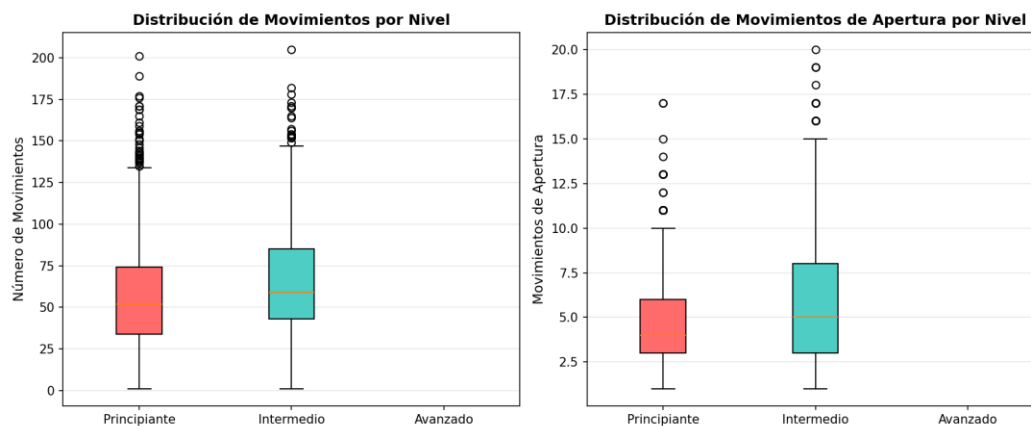


Figura 3 - Distribución de movimientos totales (turns) y de apertura (opening_ply) por nivel.

3.4 Forma de terminar la partida por nivel

La distribución de la forma de terminar la partida varia con el nivel. Los jugadores avanzados terminan más partidas por mate, mientras que los principiantes presentan una mayor proporción de outoftime, lo que refleja una gestión del tiempo menos eficiente.

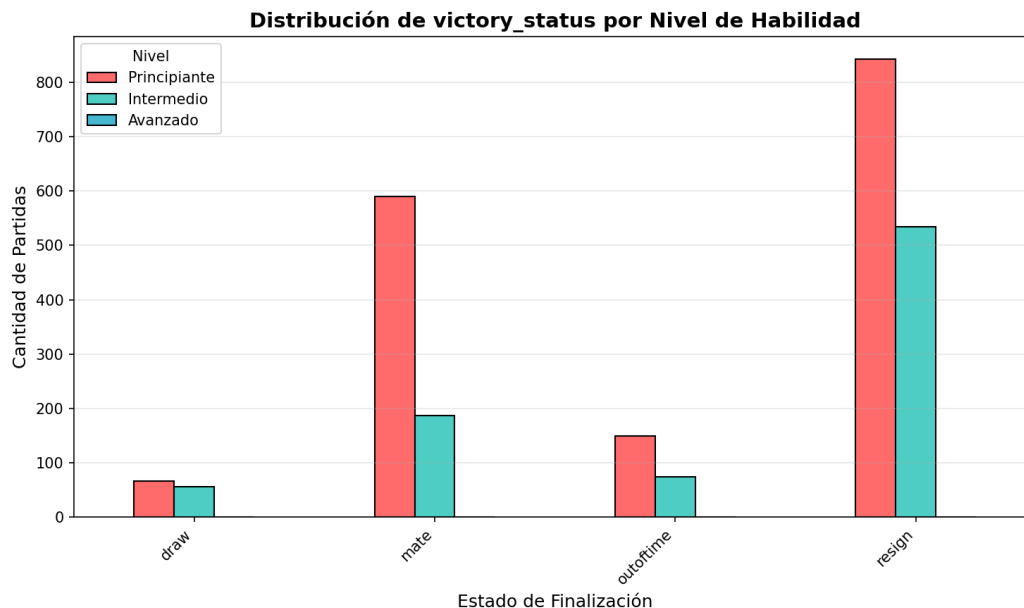


Figura 4 - Distribución de victory_status por nivel de habilidad.

3.5 Matriz de correlación

La matriz de correlación de Pearson entre las variables numéricas revela una correlación muy alta entre white_rating y black_rating (el sistema de emparejamiento une jugadores de nivel similar). Las features turns y opening_ply presentan correlaciones moderadas con los ratings. Estas variables se analizan en EDA por contexto, aunque no se incluyen como entrada del modelo.

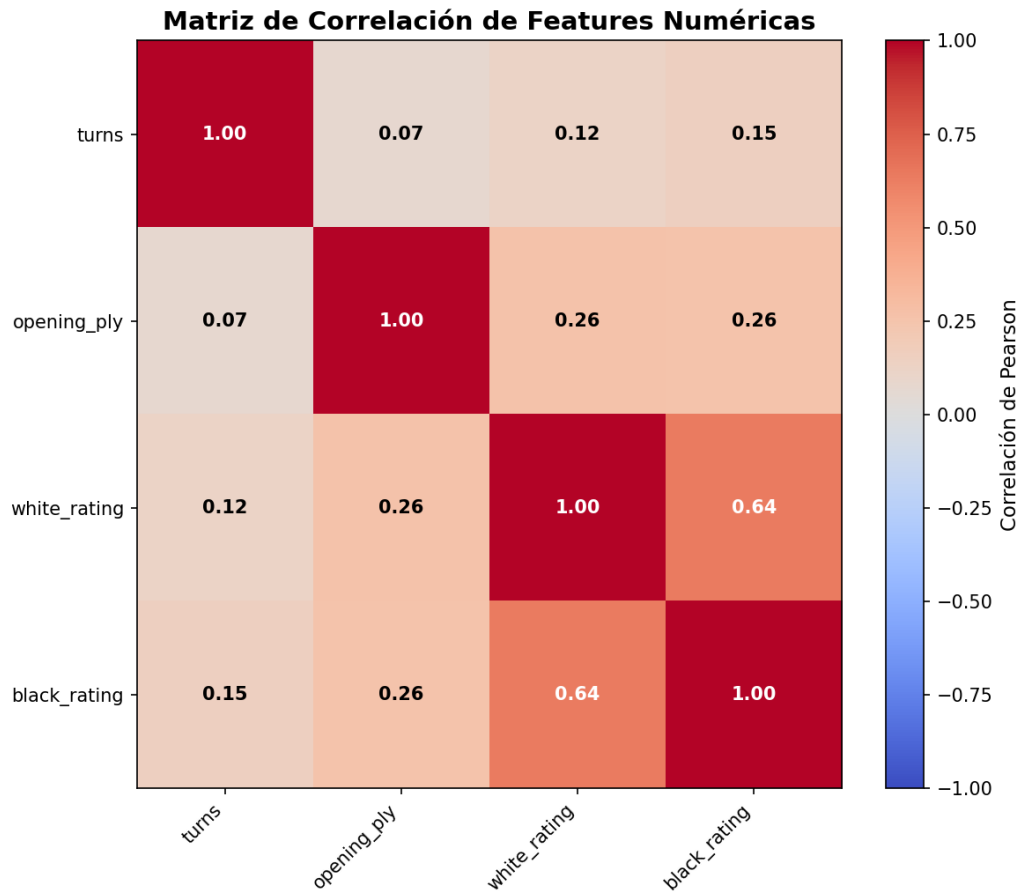


Figura 5 - Matriz de correlación de Pearson entre features numéricas.

3.6 Control de tiempo por nivel

Los boxplots del tiempo base muestran que los jugadores de mayor nivel tienden a jugar con controles de tiempo más largos (partidas de ritmo clásico), mientras que los principiantes participan más en partidas de bullet o blitz.

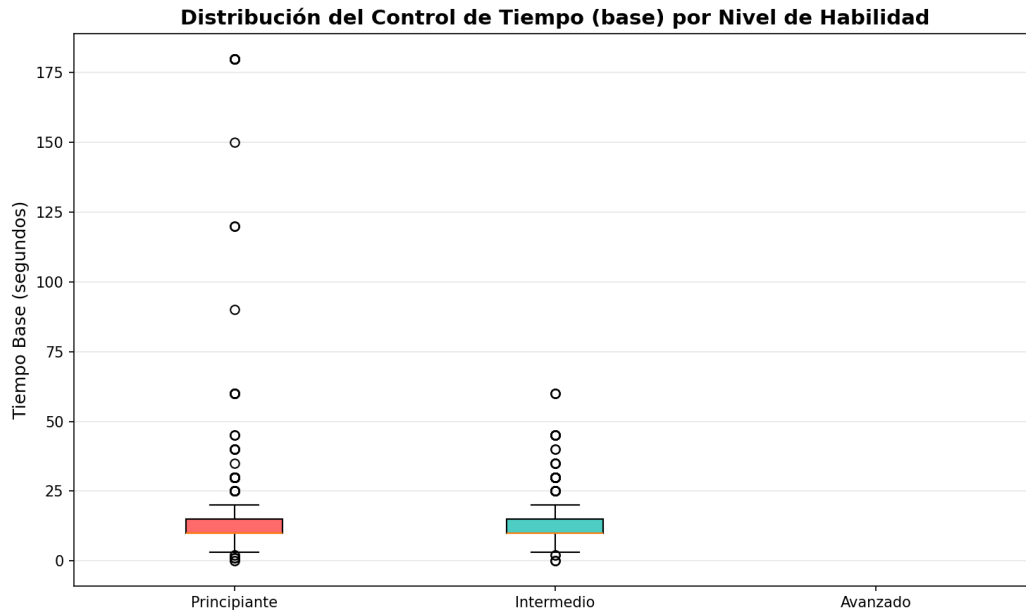


Figura 6 - Distribución del tiempo base del control de tiempo por nivel de habilidad.

En conjunto, el análisis exploratorio confirma que existe señal útil en variables como duración de partida, apertura, tipo de finalización y control de tiempo, aunque con solapamientos relevantes entre niveles. Asimismo, se observa un desbalance entre clases en el escenario binario, lo que motiva el uso de técnicas de balanceo y de métricas que no se vean dominadas por la clase mayoritaria. Estas observaciones guían las decisiones del capítulo siguiente: codificación de variables categóricas, escalado de numéricas y balanceo del conjunto de entrenamiento.

4. Preprocesado de datos

4.1 Feature engineering

A partir de las columnas originales del dataset se construyeron 17 features que alimentan los modelos. El proceso de extracción se describe a continuación:

Feature	Tipo	Transformación	Descripción
turns	Numérico	StandardScaler	Número total de movimientos
opening_ply	Numérico	StandardScaler	Movimientos de apertura reconocidos
victory_status	Categorico (4 vals)	One-Hot (4 cols)	mate / resign / outoftime / draw
winner	Categorico (3 vals)	One-Hot (3 cols)	white / black / draw
rated	Binario	StandardScaler	Partida puntuada para ranking
base_time	Numérico	StandardScaler	Tiempo base extraído de increment_code
increment	Numérico	StandardScaler	Incremento por jugada extraído de increment_code
opening_eco	Categorico (5 fam.)	One-Hot (5 cols)	Familia ECO: A, B, C, D, E

Tabla 3 - Features construidas y transformaciones aplicadas (total: 17 columnas).

La columna increment_code tiene el formato "base+incremento" (por ejemplo "600+5"). Se parseo manualmente para extraer dos features numericas independientes: el tiempo base en segundos y el incremento por jugada. La columna opening_eco se agrupo por familia (primera letra del código ECO) para reducir la cardinalidad.

4.2 Normalización

Las variables numéricas continuas se normalizan con StandardScaler (media 0, desviación estándar 1). Los scalers se ajustan exclusivamente sobre el conjunto de entrenamiento y se aplican sin reajuste sobre validación y test, evitando la fuga de información (data leakage). Las variables binarias y one-hot ya están en rango [0, 1] y no requieren normalización adicional.

4.3 Balanceo de clases con SMOTE

El conjunto de entrenamiento presenta un desbalance de aproximadamente 66 % / 34 % entre Principiantes e Intermedios. Entrenar directamente sobre datos desbalanceados tiende a que el modelo maximice la clase mayoritaria, perjudicando el recall de la clase minoritaria (Intermedios).

Para mitigar este problema se aplica SMOTE (Synthetic Minority Over-sampling Technique) exclusivamente sobre el conjunto de entrenamiento. SMOTE genera muestras sintéticas de la clase minoritaria interpolando entre vecinos cercanos en el espacio de features (k=5), en lugar de simplemente duplicar muestras existentes, lo que reduce el riesgo de sobreajuste por repetición.

Resultado: el conjunto de entrenamiento pasa de 1.500 muestras desbalanceadas a 2.000 muestras equilibradas al 50 % por clase. Los conjuntos de validación y test no se modifican, preservando la distribución real para una evaluación honesta.

5. Modelos de clasificación

En este capítulo se describen los cuatro modelos de clasificación implementados y evaluados, así como el ensemble construido a partir de ellos. La selección cubre un espectro amplio de enfoques: desde modelos lineales hasta métodos de conjunto, pasando por redes neuronales y máquinas de vectores soporte. Dos de los modelos (Regresión Logística y Red Neuronal) se han implementado desde cero utilizando exclusivamente NumPy, con el objetivo de demostrar comprensión profunda de los algoritmos subyacentes. Los otros dos (SVM y Random Forest) emplean la librería scikit-learn, lo que permite aprovechar implementaciones optimizadas y facilitar la búsqueda de hiperparámetros mediante GridSearchCV.

En todos los casos se aplica ponderación de clases durante el entrenamiento para compensar el desbalance existente en el conjunto de datos (66 % Principiantes / 34 % Intermedios).

5.1 Regresión logística One-vs-All (implementación NumPy)

La Regresión Logística es un modelo lineal que estima la probabilidad de que una muestra pertenezca a una clase determinada. Para extender su aplicación al escenario multiclase se adopta la estrategia One-vs-All (OvA): se entrena un clasificador binario independiente por cada clase, de modo que cada uno aprende a distinguir su clase del resto. La predicción final se asigna a la clase cuyo clasificador devuelve la mayor probabilidad.

El modelo se optimiza mediante descenso de gradiente, incorporando regularización L2 para penalizar pesos excesivamente grandes y reducir el riesgo de sobreajuste. Para garantizar la estabilidad numérica, las activaciones se recortan antes de calcular el logaritmo y los pesos se inicializan con valores próximos a cero.

Decisiones de implementación. El modelo se ha implementado íntegramente con NumPy, sin hacer uso de scikit-learn para el ajuste de parámetros. Se calculan pesos de clase balanceados mediante `compute_class_weight` para que los errores sobre la clase minoritaria (Intermedios) penalicen proporcionalmente más durante el entrenamiento.

Búsqueda de hiperparámetros. Se exploró un grid manual de 12 combinaciones sobre la tasa de aprendizaje ($\text{learning_rate} \in \{0.01, 0.1, 0.5\}$) y el coeficiente de regularización ($\text{lambda_reg} \in \{0.0, 0.01, 0.1, 1.0\}$), fijando el número de iteraciones en 1.000. El criterio de selección fue el accuracy sobre el conjunto de validación. La Figura 7 muestra la curva de validación de `lambda_reg`, que permite identificar el rango de regularización con mejor equilibrio entre sesgo y varianza. La Figura 8 presenta los coeficientes aprendidos por el clasificador de la clase Intermedio.

Hiperparámetro	Valores explorados
<code>learning_rate</code>	0.01 0.1 0.5
<code>lambda_reg</code>	0.0 0.01 0.1 1.0
<code>n_iterations</code>	1.000 (fijo)

Tabla 4 - Grid de hiperparámetros para Regresión Logística OvA.

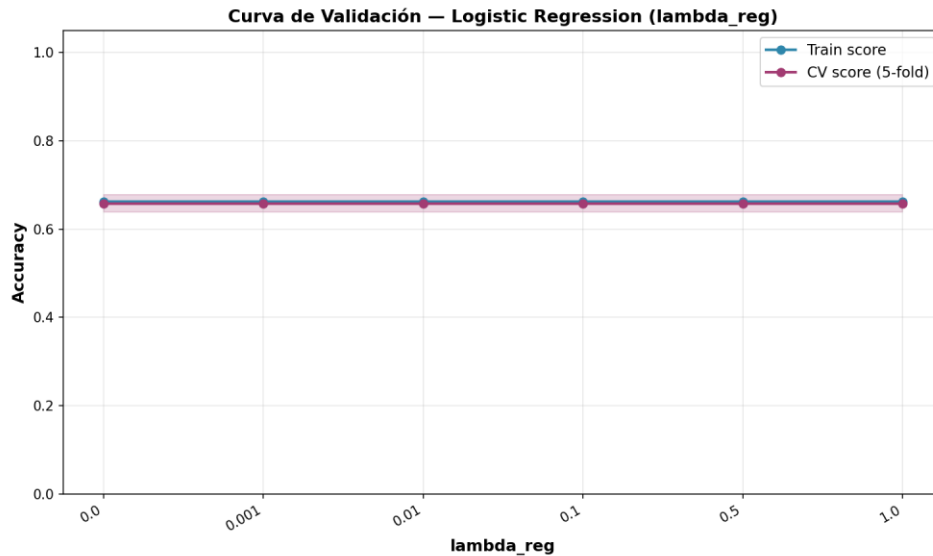


Figura 7 - Curva de validación de λ_{reg} para Regresión Logística (5-fold CV).

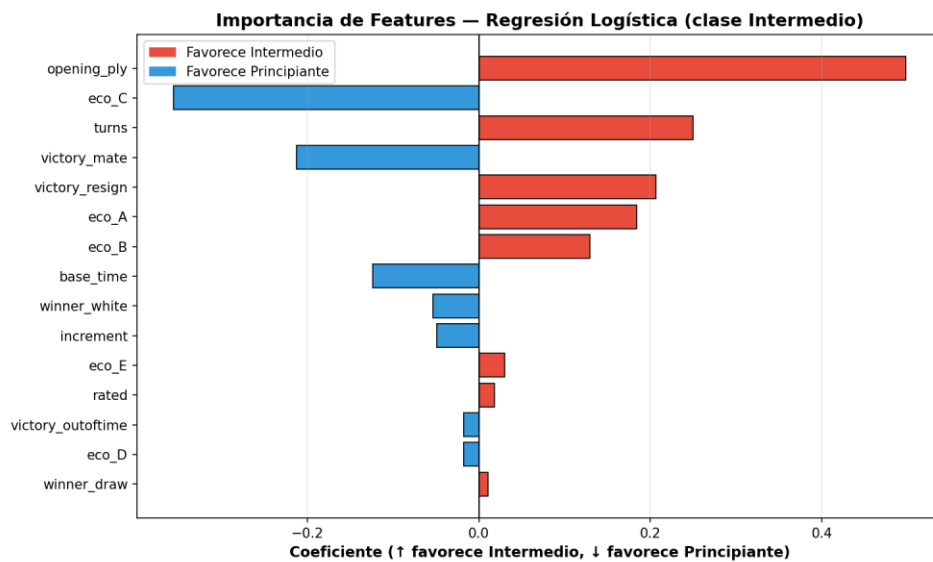


Figura 8 - Importancia de features según los coeficientes de la Regresión Logística (clase Intermedio).

5.2 Red neuronal feedforward (implementación NumPy)

La Red Neuronal Feedforward (también denominada Perceptrón Multicapa o MLP) es un modelo no lineal capaz de aprender representaciones jerárquicas de los datos a través de capas sucesivas de transformaciones. La arquitectura implementada consta de una capa de entrada de 17 neuronas (correspondiente al número de features), una o varias capas ocultas con activación ReLU, y una capa de salida con activación Softmax que devuelve una distribución de probabilidad sobre las clases.

La función ReLU introduce no linealidad de forma computacionalmente eficiente, mientras que Softmax en la salida garantiza que las probabilidades de clase sumen uno y sean directamente interpretables. La función de pérdida empleada es la entropía cruzada categórica con regularización L2, ponderada por los pesos de clase para mitigar el desbalance.

Backpropagation. El algoritmo de retropropagación se ha implementado manualmente, calculando los gradientes capa a capa mediante la regla de la cadena. La derivada combinada de Softmax y entropía cruzada simplifica la expresión del gradiente en la capa de salida, y los gradientes de las capas ocultas se propagan multiplicando por la derivada de ReLU. Los pesos se inician siguiendo la inicialización de He, recomendada para redes con activaciones ReLU, ya que escala la varianza de los pesos en función del número de entradas de cada capa para evitar la saturación o la desaparición del gradiente.

Entrenamiento y Early Stopping. El entrenamiento se realiza mediante Minibatch SGD, dividiendo el conjunto de entrenamiento en lotes de tamaño configurable antes de cada época. Se implementa Early Stopping monitorizando la pérdida en el conjunto de validación: si no mejora durante un número de épocas consecutivas (paciencia = 20), el entrenamiento se detiene y se restauran los pesos correspondientes a la mejor época registrada. Este mecanismo evita el sobreajuste sin necesidad de fijar a priori el número de épocas. La Figura 9 muestra la evolución de la pérdida y el accuracy en entrenamiento y validación a lo largo de las épocas, con la línea de Early Stopping señalada.

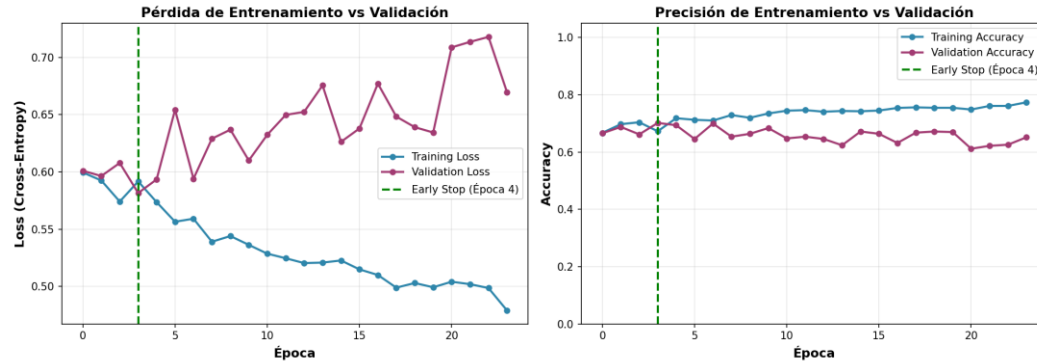


Figura 9 - Evolución de la pérdida y el accuracy por época en train y validación. La línea verde marca el punto de Early Stopping.

Búsqueda de hiperparámetros. Se exploró un grid de 54 combinaciones sobre la arquitectura de capas ocultas ($\{[32], [64, 32], [128, 64, 32]\}$), la tasa de aprendizaje ($\{0.001, 0.01, 0.1\}$), el coeficiente de regularización ($\{0.0, 0.001, 0.01\}$) y el tamaño de lote ($\{32, 64\}$). La Figura 10 muestra la curva de validación de la tasa de aprendizaje, que ilustra el efecto de este hiperparámetro sobre el rendimiento en validación cruzada.

Hiperparámetro	Valores explorados
hidden_layers	[32] [64, 32] [128, 64, 32]
learning_rate	0.001 0.01 0.1
lambda_reg	0.0 0.001 0.01
batch_size	32 64
max_epochs / patience	200 / 20 (fijos)

Tabla 5 - Grid de hiperparámetros para la Red Neuronal.

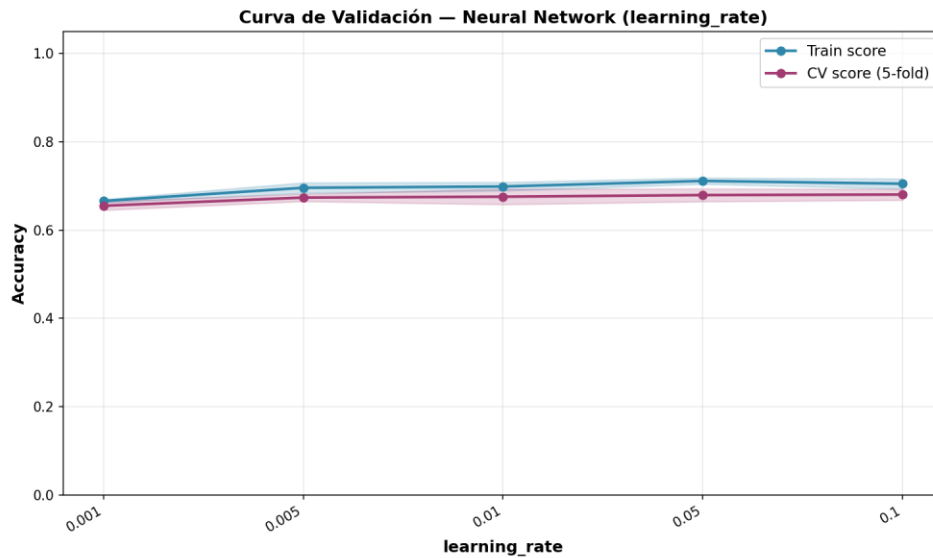


Figura 10 - Curva de validación del learning_rate para la Red Neuronal (5-fold CV).

5.3 Support vector machine con kernel RBF (scikit-learn)

La Máquina de Vectores Soporte (SVM) es un clasificador que busca el hiperplano de separación con el mayor margen posible entre clases. Cuando los datos no son linealmente separables en el espacio original de features, el denominado truco del kernel permite operar implícitamente en un espacio de mayor dimensión sin necesidad de calcularlo explícitamente. En este proyecto se emplea el kernel RBF (Radial Basis Function), que mide la similitud entre dos muestras en función de su distancia euclidiana y es especialmente adecuado cuando no se conoce a priori la estructura de los datos.

El parámetro C controla el equilibrio entre maximizar el margen de separación y minimizar los errores de clasificación en entrenamiento: valores bajos de C generan márgenes amplios, pero toleran más errores, mientras que valores altos ajustan más la frontera a los datos con riesgo de sobreajuste. El parámetro gamma regula la influencia de cada muestra individual sobre la frontera de decisión.

Se utiliza `class_weight='balanced'` para que el modelo penalice más los errores sobre la clase minoritaria, compensando el desbalance sin necesidad de modificar el conjunto de datos.

Búsqueda de hiperparámetros. Se aplicó GridSearchCV con validación cruzada de 5 pliegues sobre 16 combinaciones: $C \in \{0.1, 1, 10, 100\}$ y $\gamma \in \{'scale', 'auto', 0.01, 0.1\}$. El criterio de selección fue el accuracy medio en los pliegues de validación.

Hiperparámetro	Valores explorados
C	0.1 1 10 100
gamma	"scale" "auto" 0.01 0.1

Tabla 6 - Grid de hiperparámetros para SVM RBF.

5.4 Random Forest (scikit-learn)

Random Forest es un método de aprendizaje por conjuntos (ensemble) que construye un gran número de árboles de decisión de forma independiente y combina sus predicciones. Cada árbol se entrena sobre una muestra aleatoria con reemplazo del conjunto de entrenamiento (bagging), y en cada nodo de división solo se considera un subconjunto aleatorio de features. Esta doble fuente de aleatoriedad reduce la correlación entre árboles, lo que disminuye la varianza del modelo final respecto a un árbol individual.

La predicción de clase se obtiene promediando las probabilidades estimadas por todos los árboles, lo que produce estimaciones suaves y bien calibradas. El modelo es además robusto frente al sobreajuste y proporciona, como subproducto, una medida de la importancia relativa de cada feature basada en la reducción de impureza que aportan en las divisiones. La Figura 11 muestra estas importancias para las 15 features más relevantes según el mejor modelo encontrado.

Al igual que con la SVM, se emplea `class_weight='balanced'` para compensar el desbalance de clases.

Búsqueda de hiperparámetros. Se aplicó GridSearchCV con validación cruzada de 5 pliegues sobre 36 combinaciones: número de árboles `n_estimators` $\in \{50, 100, 200\}$, profundidad máxima `max_depth` $\in \{\text{None}, 5, 10, 20\}$ y mínimo de muestras para dividir un nodo `min_samples_split` $\in \{2, 5, 10\}$.

Hiperparámetro	Valores explorados
n_estimators	50 100 200
max_depth	None 5 10 20
min_samples_split	2 5 10

Tabla 7 - Grid de hiperparámetros para Random Forest.

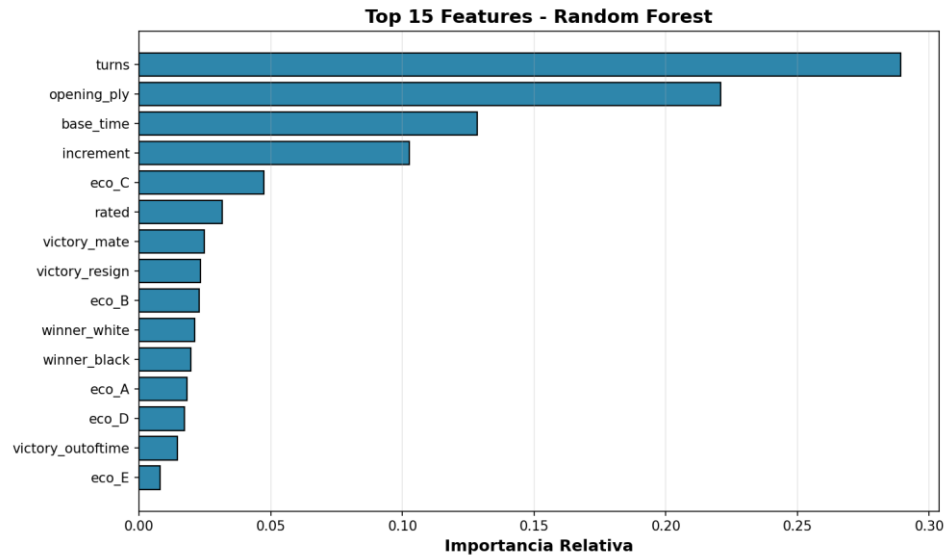


Figura 11 - Importancia relativa de features según el Random Forest.

5.5 Ensemble de votación suave (implementación NumPy)

Como técnica complementaria se construye un Ensemble de votación suave (soft voting) que combina las predicciones probabilísticas de los cuatro modelos anteriores. A diferencia de la votación por mayoría, que solo tiene en cuenta la clase predicha, la votación suave pondera las probabilidades estimadas por cada modelo antes de tomar la decisión final. Esto aprovecha la calibración probabilística de cada clasificador y tiende a producir predicciones más robustas cuando los modelos individuales cometen errores en muestras diferentes.

En este caso, cada modelo contribuye con un peso proporcional a su ROC-AUC en el conjunto de prueba, de modo que los modelos con mayor capacidad discriminativa tienen más influencia en la decisión final. La predicción final se asigna a la clase con mayor probabilidad ponderada agregada.

El principal beneficio esperado de esta estrategia es la reducción de la varianza respecto a cualquier modelo individual, suavizando los comportamientos extremos y corrigiendo los errores específicos de cada clasificador.

6. Evaluación y comparación de modelos

6.1 Métricas de evaluación

Dado el desbalance de clases presente en el conjunto de prueba, 330 partidas de Principiante frente a 170 de Intermedio (66 % / 34 %), el accuracy por sí solo no constituye una métrica de evaluación suficiente. Un clasificador trivial que predijera siempre la clase mayoritaria alcanzaría un 66 % de accuracy sin haber aprendido ningún patrón. Por este motivo, las métricas principales empleadas en este estudio son las siguientes:

- F1-Score macro: media armónica de precisión y recall, promediada por igual sobre ambas clases. Penaliza de forma equilibrada el mal rendimiento en la clase minoritaria y es la métrica de referencia para comparar modelos.
- ROC-AUC: área bajo la curva ROC. Mide la capacidad discriminativa del modelo de forma independiente al threshold de decisión, lo que lo hace útil para comparar modelos sin que el umbral de clasificación condicione el resultado.
- Precision y Recall por clase: permiten detectar sesgos hacia la clase mayoritaria y evaluar el comportamiento específico de cada modelo en la clase minoritaria (Intermedio).
- Matriz de confusión: proporciona visibilidad completa sobre los tipos de error cometidos, distinguiendo entre falsos positivos y falsos negativos por clase.

6.2 Validación cruzada estratificada 5-Fold

Para obtener una estimación robusta del rendimiento generalizable, se aplicó validación cruzada estratificada de 5 pliegues sobre el conjunto train+validación combinada. La estratificación garantiza que la proporción de clases se preserve en cada pliegue, evitando que alguna división favorezca artificialmente a un modelo.

Para los modelos implementados con NumPy (Regresión Logística y Red Neuronal), que no son compatibles con `cross_val_score` de scikit-learn, se implementó un bucle de CV manual que replica el mismo protocolo. En el caso de la Red Neuronal, se reserva adicionalmente un 20 % del pliegue de entrenamiento como conjunto de validación interna para el mecanismo de Early Stopping, preservando así la coherencia del proceso de entrenamiento en cada fold.

6.3 Curvas ROC

Las curvas ROC (Figuras 12–16) permiten evaluar la capacidad discriminativa de cada modelo con independencia del threshold de decisión. El eje vertical representa la tasa de verdaderos positivos (recall de Intermedios) y el eje horizontal la tasa de falsos positivos, de modo que un modelo perfecto ocuparía la esquina superior izquierda y un clasificador aleatorio seguiría la diagonal.

La Red Neuronal obtiene el AUC más alto (0.737), seguida del Ensemble (0.726), el Random Forest (0.711), la Regresión Logística (0.706) y, en último lugar, la SVM (0.646). Visualmente, las curvas

de NN, RF y LR presentan un perfil similar, con un ascenso inicial pronunciado que refleja que, para umbrales bajos, estos modelos identifican bien la clase Intermedio antes de empezar a acumular falsos positivos. La curva de la SVM, en cambio, se mantiene más próxima a la diagonal especialmente en la región de baja tasa de falsos positivos, lo que evidencia una discriminación más limitada.

Es importante destacar que el hecho de que la NN sea el mejor modelo según el AUC no significa que sea el mejor a threshold 0.5. El AUC refleja el comportamiento integrado en todos los umbrales posibles, pero si a 0.5 el modelo está sesgado hacia la clase mayoritaria, su rendimiento práctico puede ser inferior al de modelos con menor AUC.

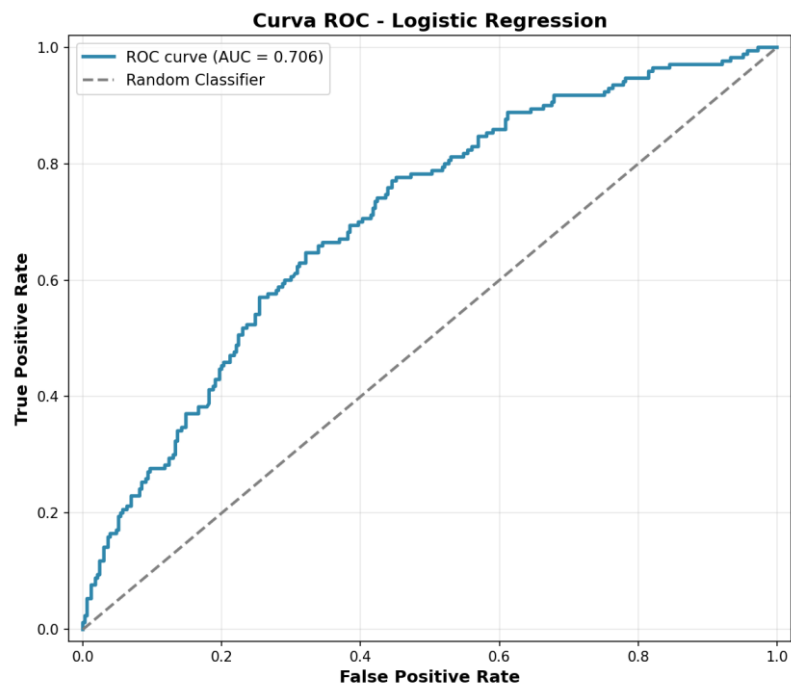


Figura 12 - Curva ROC - Regresión logística (AUC = 0.706).

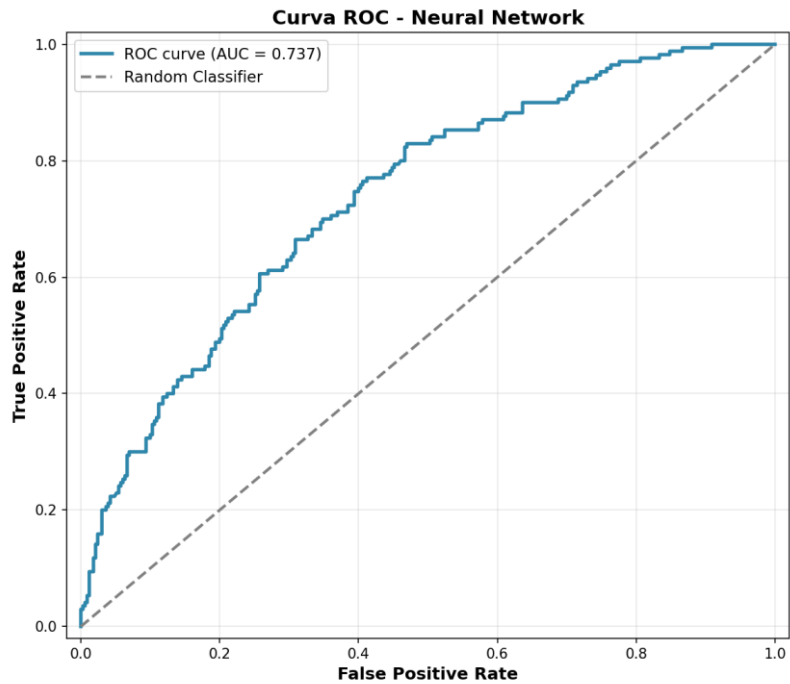


Figura 13 - Curva ROC - Red Neuronal (AUC = 0.737).

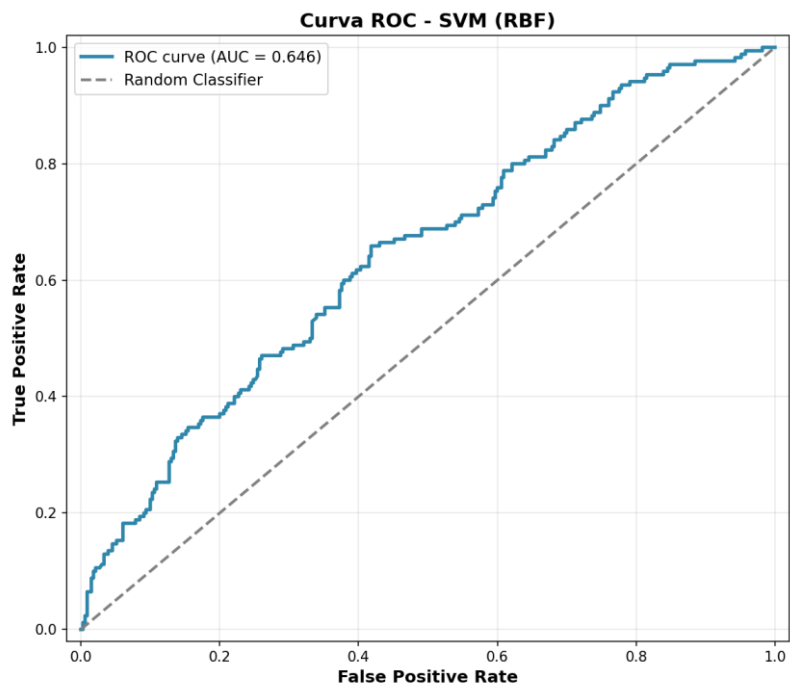


Figura 14 - Curva ROC - SVM RBF (AUC = 0.646).

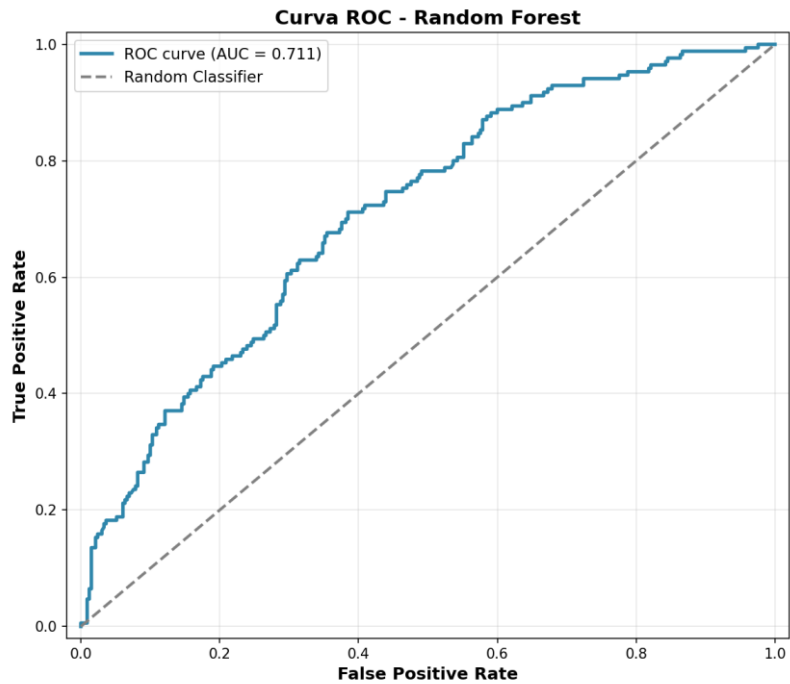


Figura 15 - Curva ROC - Random Forest (AUC = 0.711).

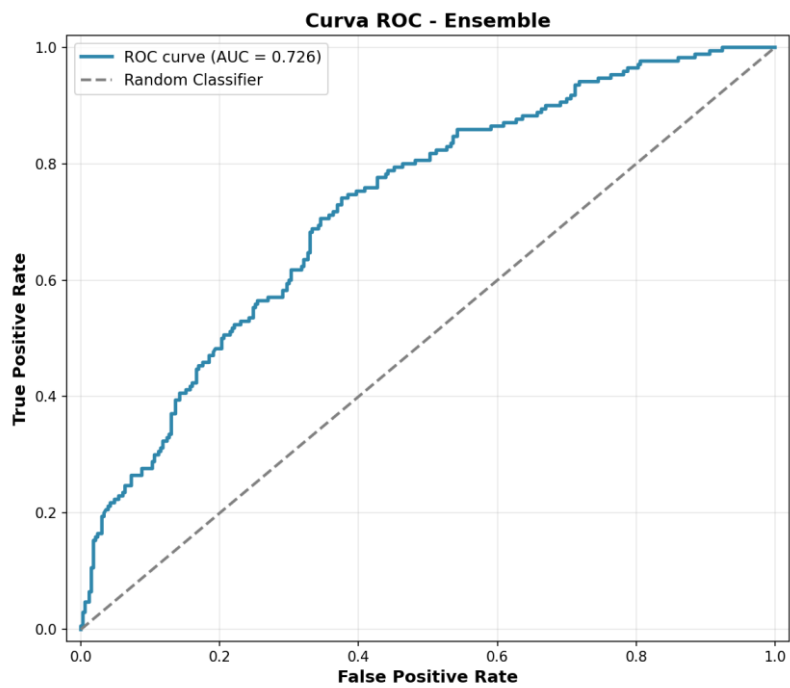


Figura 16 - Curva ROC - Ensemble Soft Voting (AUC = 0.726).

6.4 Matrices de confusión

Las matrices de confusión (Figuras 17–21) revelan el patrón de errores de cada modelo sobre el conjunto de prueba con threshold por defecto (0.5), y permiten identificar comportamientos que las métricas agregadas ocultan.

El dato más llamativo es el de la Red Neuronal (Figura 18): con 277 Principiantes correctamente clasificados siendo el mejor resultado entre todos los modelos en esa clase, solo clasifica correctamente 73 de los 170 Intermedios, dejando 97 sin detectar. Este comportamiento indica que el modelo, pese a su mayor capacidad expresiva, aprendió a predecir mayoritariamente la clase Principiante, probablemente porque la señal de las features no es suficientemente rica para que la no linealidad de la red aporte ventaja real en este problema. El resultado es un modelo con alta accuracy pero escasa utilidad práctica para la detección de Intermedios.

En el extremo opuesto se sitúa la Regresión Logística (Figura 17), que con 107 Intermedios correctamente detectados ($\text{recall} = 0.63$) ofrece el mejor equilibrio entre clases a threshold 0.5, a costa de 105 falsos positivos en la clase Principiante. Este comportamiento es coherente con el hecho de que la LR aplica pesos de clase explícitos durante el entrenamiento y tiene una frontera de decisión lineal que no tiende a colapsar hacia la clase mayoritaria de la misma manera que la NN.

La SVM (Figura 19) muestra una distribución de errores intermedia: 90 Intermedios detectados frente a 110 falsos negativos. Aunque su accuracy global es el peor (0.620), su recall de Intermedios a threshold 0.5 (0.53) no es el peor, lo que sugiere que la SVM sacrifica precisión en Principiantes a cambio de un comportamiento algo más equilibrado entre clases.

El Random Forest (Figura 20) presenta la peor distribución relativa: pese a su accuracy razonable (0.676), solo detecta 78 de los 170 Intermedios ($\text{recall} = 0.46$), con 92 falsos negativos. El modelo es el más conservador en la predicción de Intermedios, lo que, combinado con su elevado training score (próximo a 1.0 en las curvas de aprendizaje), apunta a un comportamiento de sobreajuste que le hace perder capacidad de generalización hacia la clase minoritaria.

El Ensemble (Figura 21) obtiene un resultado intermedio a threshold 0.5, con 89 Intermedios correctos. Sin embargo, su principal valor no reside en este umbral sino en su comportamiento al ajustar el threshold, como se analiza en la sección siguiente.

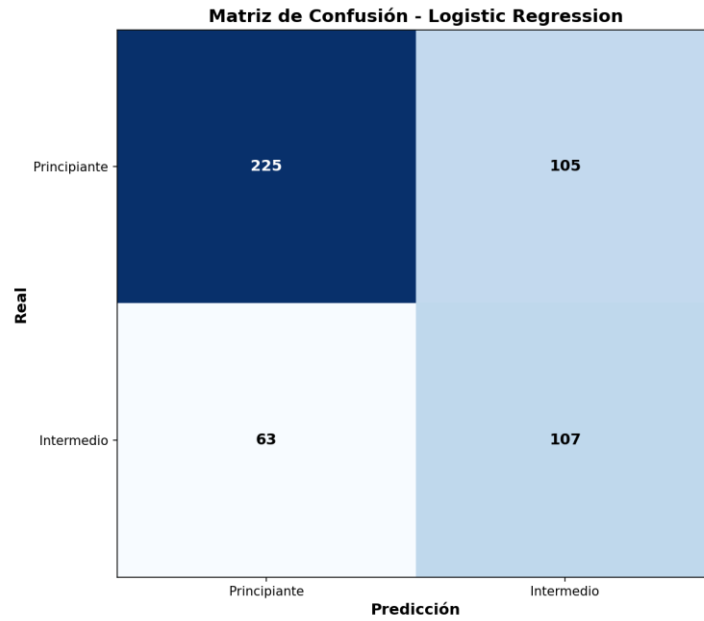


Figura 17 - Matriz de confusión - Regresión logística (threshold=0.5).

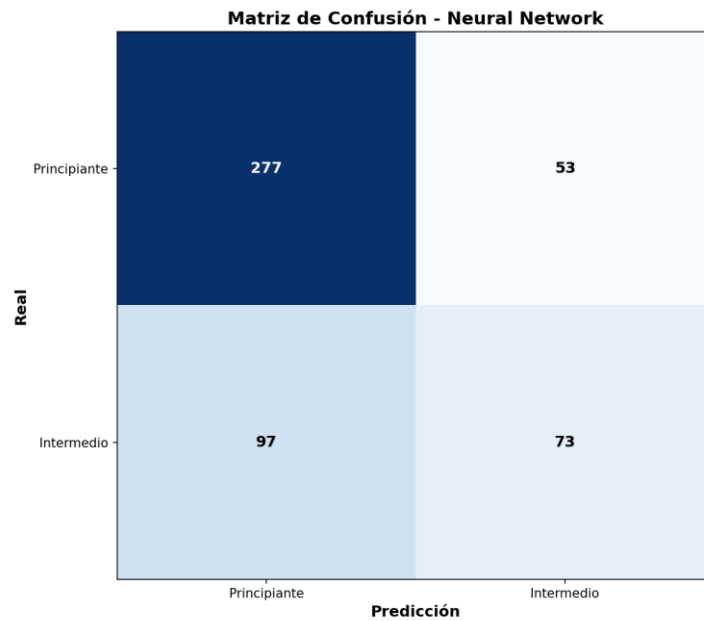


Figura 18 - Matriz de confusión - Red Neuronal (threshold=0.5).

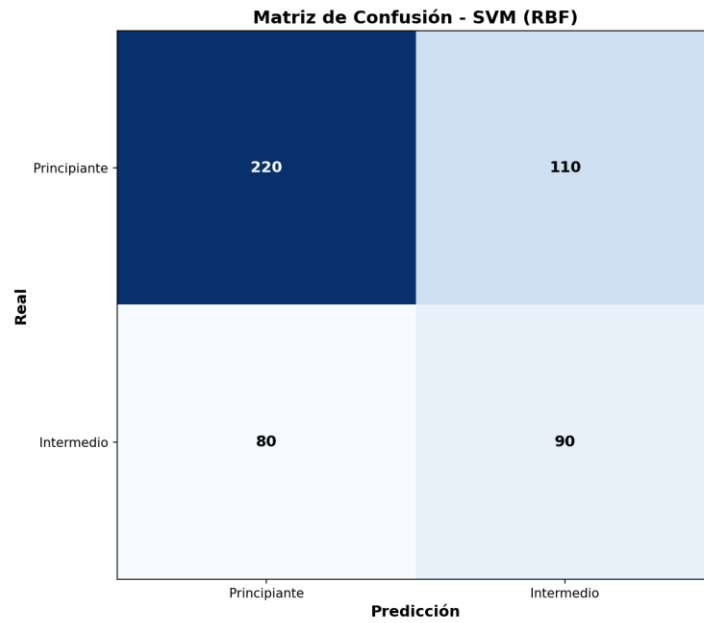


Figura 19 - Matriz de confusión - SVM RBF (threshold=0.5).

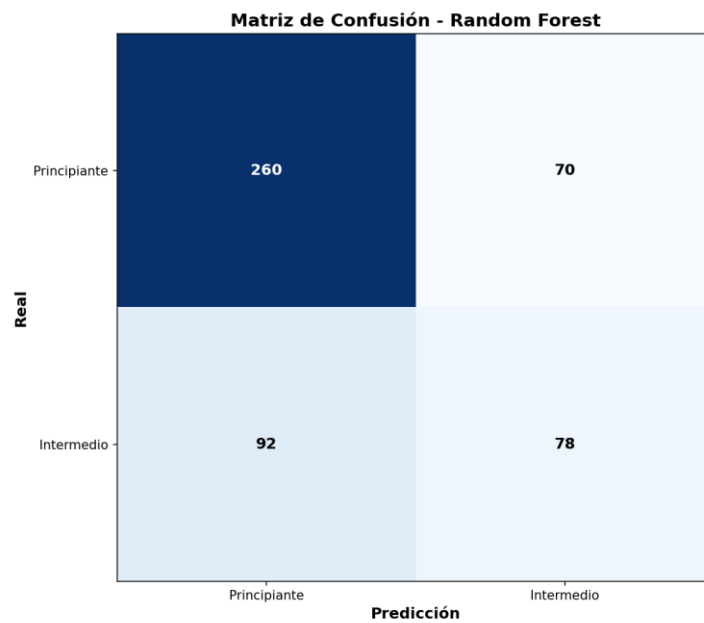


Figura 20 - Matriz de confusión - Random Forest (threshold=0.5).

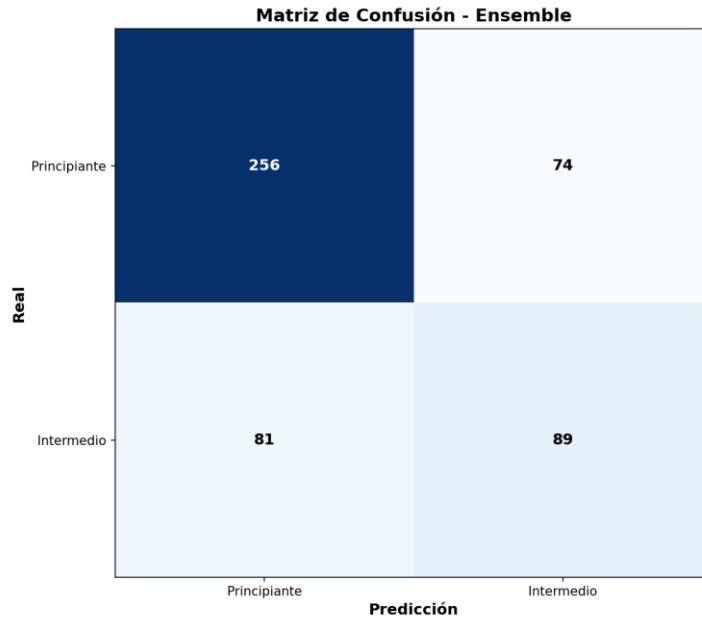


Figura 21 - Matriz de confusión - Ensemble Soft Voting (threshold=0.5).

6.5 Resultados con threshold por defecto (0.5)

La Tabla 8 recoge las métricas de los cinco modelos sobre el conjunto de prueba con el threshold de decisión por defecto (0.5).

Modelo	Accuracy	F1 Macro	ROC-AUC	Recall Intermedio
Logistic Regression	0.664	0.644	0.706	0.63
Neural Network	0.700	0.640	0.737	0.43
SVM (RBF)	0.620	0.592	0.646	0.53
Random Forest	0.676	0.627	0.711	0.46
Ensemble	0.690	0.651	0.726	0.52

Tabla 8 - Resultados de los cinco modelos en el conjunto de prueba (threshold = 0.5).

Los resultados ponen de manifiesto una tensión recurrente entre accuracy y recall de la clase minoritaria. La Red Neuronal lidera en accuracy (0.700) y en ROC-AUC (0.737), pero presenta el peor recall de Intermedios (0.43): clasifica correctamente solo 73 de los 170 jugadores Intermedios del conjunto de prueba. En el otro extremo, la Regresión Logística, con un accuracy 3,6 puntos inferior, detecta correctamente 107 Intermedios, lo que la convierte en el modelo más equilibrado a este threshold.

El Ensemble obtiene el mejor F1 macro (0.651), situándose por encima de cualquier modelo individual en esta métrica. Esto confirma que la combinación ponderada de probabilidades permite suavizar los sesgos particulares de cada clasificador, aunque no elimina completamente el problema del recall bajo en Intermedios.

La SVM es el modelo con peores resultados globales: su AUC de 0.646 indica una capacidad discriminativa limitada y su F1 macro de 0.592 es el más bajo. A diferencia de los demás modelos, la SVM no parece beneficiarse de la misma forma de la ponderación de clases con el kernel RBF en este espacio de features.

6.6 Ajuste de threshold para mejorar el recall de intermedios

El threshold de 0.5 es un valor convencional que no tiene por qué ser el óptimo en un problema con desbalance de clases. Reducir el umbral de decisión para la clase Intermedio implica clasificar como Intermedio muestras con probabilidades más bajas, lo que aumenta el recall a costa de reducir la precisión. Para cada modelo, se realizó una búsqueda del threshold que maximiza el F1-Score de la clase Intermedio sujeto a una precisión mínima del 40 %, explorando el rango [0.10, 0.70] con 121 pasos uniformes.

Modelo	Threshold	Accuracy	F1 Macro	Recall Interm.	Prec. Interm.
LR	0.460	0.628	0.624	0.77	0.47
NN	0.275	0.632	0.630	0.82	0.48
SVM	0.435	0.606	0.596	0.66	0.45
RF	0.275	0.570	0.570	0.88	0.43
Ensemble	0.410	0.662	0.653	0.74	0.50

Tabla 9 - Resultados con threshold ajustado para maximizar recall de Intermedios.

El ajuste de threshold produce mejoras sustanciales en el recall de Intermedios para todos los modelos. El caso más extremo es el Random Forest con threshold 0.275, que alcanza un recall de 0.88, detectando 150 de los 170 Intermedios, pero a un coste muy alto: su accuracy cae hasta 0.570 y su precisión de Intermedios se sitúa en el límite mínimo aceptable (0.43), lo que implica un gran número de Principiantes clasificados erróneamente. Este resultado ilustra bien el trade-off precision-recall inherente al ajuste de threshold: se puede forzar el recall a valores muy altos, pero el coste en falsos positivos puede volverse inaceptable.

La Figura 22 muestra la matriz de confusión del modelo recomendado, el Ensemble con threshold = 0.410, que representa el mejor equilibrio entre recall y precisión: 126 Intermedios correctamente clasificados (frente a los 89 con threshold 0.5, un incremento del 41,6 %) y 44 falsos negativos, manteniendo una precisión de Intermedios del 50 % y un F1 macro de 0.653.

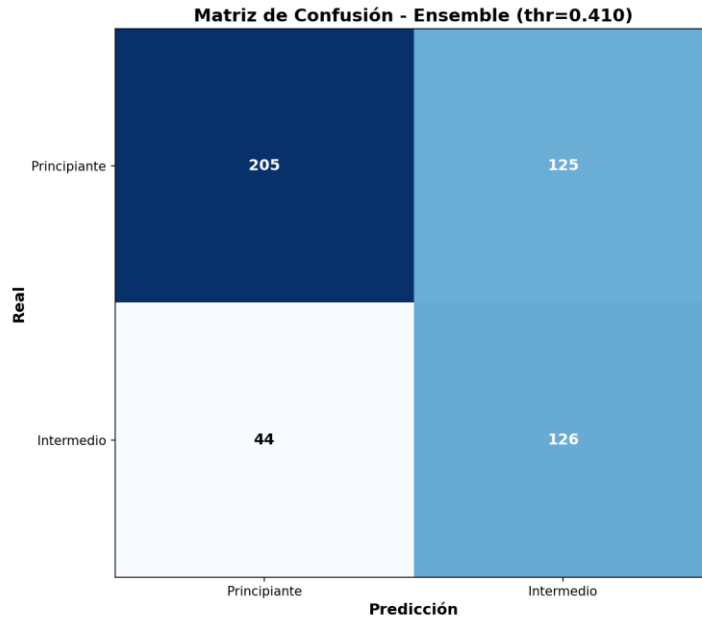


Figura 22 - Matriz de confusión - Ensemble con threshold=0.410 (modelo recomendado).

6.7 Análisis del comportamiento de generalización

Las curvas de aprendizaje de las Figuras 23 y 24 complementan el análisis de resultados mostrando cómo evoluciona el rendimiento de SVM y Random Forest en función del tamaño del conjunto de entrenamiento, y permiten diagnosticar la presencia de overfitting como factor explicativo de los resultados observados.

En ambos modelos se aprecia un gap persistente entre el training score y el score de validación cruzada, indicativo de que los modelos memorizan patrones del conjunto de entrenamiento que no generalizan completamente a datos nuevos. Sin embargo, la naturaleza del overfitting difiere entre ambos.

En el caso de la SVM (Figura 23), el training score parte de valores próximos a 0.90 con pocos datos y descende progresivamente conforme aumenta el tamaño del conjunto, mientras que el score de validación cruzada asciende desde aproximadamente 0.58 hasta situarse en torno a 0.68–0.70. La tendencia convergente de ambas curvas indica que la SVM se beneficia de disponer de más datos, y que con un dataset más amplio el gap podría reducirse aún más. No obstante, el nivel al que convergen ambas curvas sigue siendo moderado, lo que confirma que la limitación no es únicamente de datos sino también de la señal disponible en las features.

El Random Forest (Figura 24) presenta un comportamiento más acusado: su training score se mantiene prácticamente constante cerca de 0.90 independientemente del tamaño del conjunto de entrenamiento, mientras que el score de validación cruzada mejora desde 0.58 hasta aproximadamente 0.75 al disponer de más datos. Esta rigidez del training score, combinada con la

mejora continua del score de validación, es un indicador clásico de sobreajuste estructural: el modelo aprende a clasificar casi perfectamente los datos de entrenamiento, pero generaliza con dificultad, especialmente en la clase minoritaria, donde los patrones son menos frecuentes. Este comportamiento es consistente con los resultados de la sección anterior: el Random Forest presenta el recall de Intermedios más bajo a threshold 0.5 (0.46) pese a un accuracy aparentemente razonable (0.676), lo que refleja precisamente esta tendencia a memorizar los ejemplos de la clase mayoritaria.

Ambas curvas muestran que ninguno de los dos modelos ha alcanzado su techo de rendimiento con las muestras de entrenamiento disponibles, ya que el score de validación cruzada sigue una tendencia ascendente sin haberse estabilizado claramente. Esto sugiere que ampliar el conjunto de datos podría traducirse en mejoras reales de generalización, y constituye uno de los argumentos más sólidos a favor de trabajar con el dataset completo como línea de mejora futura.

Desde la perspectiva del análisis comparativo, el overfitting observado en SVM y Random Forest refuerza la idoneidad del Ensemble como modelo recomendado: al combinar las probabilidades de cuatro clasificadores con distintos sesgos y distintos grados de sobreajuste, el Ensemble reduce la varianza de la predicción final y produce un comportamiento más robusto que cualquier modelo individual, tal y como se refleja en su mayor F1 macro y en la estabilidad de su curva ROC.

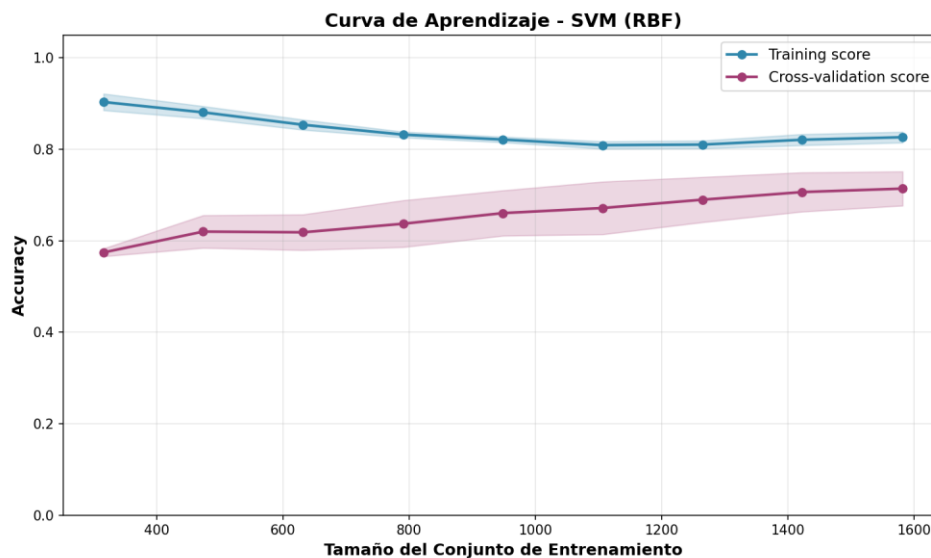


Figura 23 - Curva de aprendizaje - SVM RBF.

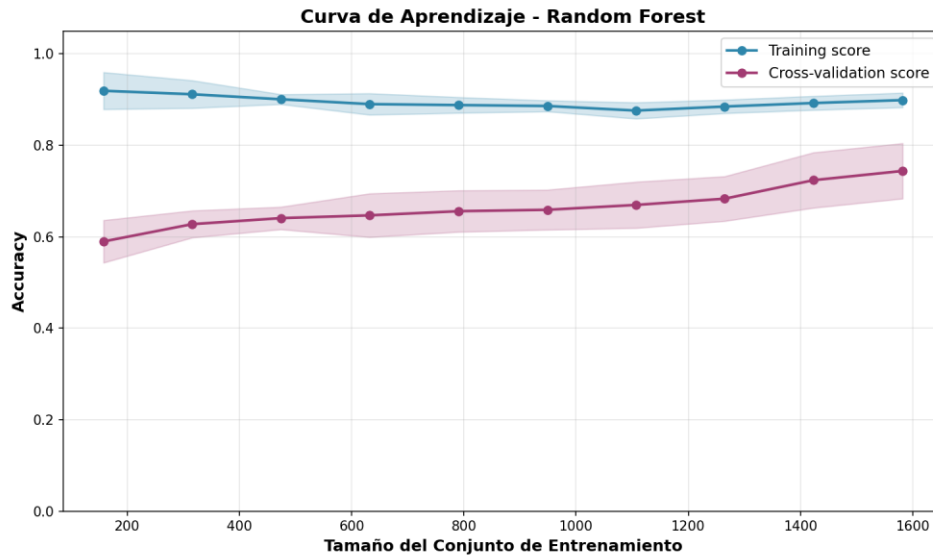


Figura 24 - Curva de aprendizaje - Random Forest.

6.8 Modelo recomendado

Tras el análisis comparativo de métricas, matrices de confusión, curvas ROC y comportamiento de generalización, el modelo recomendado para este escenario es el Ensemble de votación suave con umbral de decisión ajustado (threshold = 0.410), cuya matriz de confusión se detalla en la Figura 22. A continuación, se presenta la comparativa global de rendimiento de los modelos base a su umbral estándar (threshold = 0.5), la cual sirve como punto de partida para la optimización matemática:

Tabla 8 - Resultados de los cinco modelos en el conjunto de prueba (threshold = 0.5)

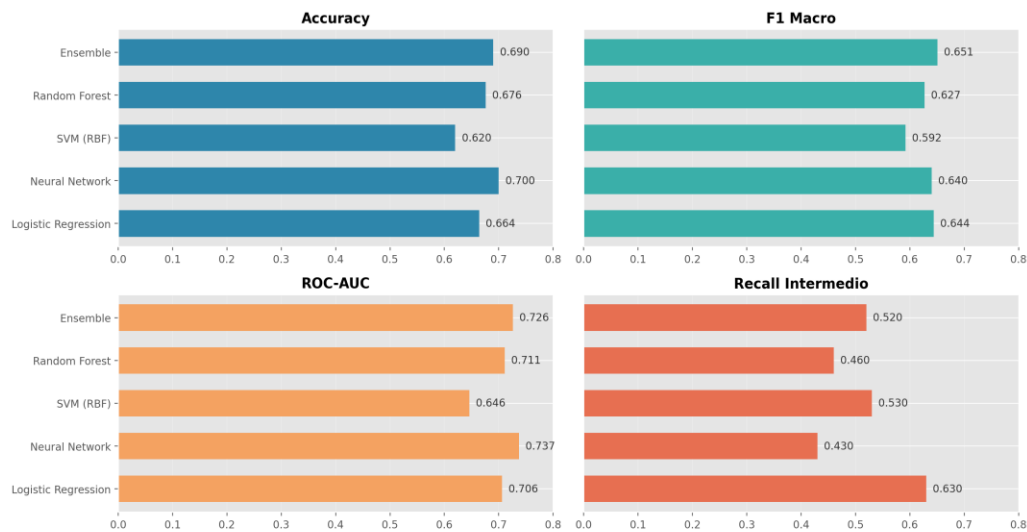


Figura 25 - Comparativa de modelos

La elección de la arquitectura de Ensamble y la posterior modificación de su punto de operación operacional específico se sustentan en los siguientes argumentos ingenieriles:

Estructura base óptima para el ajuste: Tal y como se observa en la Figura 25, operando bajo el umbral por defecto (threshold = 0.5), el Ensemble ya se posiciona como una de las opciones más sólidas y equilibradas del estudio, alcanzando un Accuracy de 0.690, un F1-Macro de 0.651 y un ROC-AUC de 0.726. Aunque la Red Neuronal muestra un ROC-AUC ligeramente superior (0.737), colapsa con mayor frecuencia hacia la clase mayoritaria debido a su bajo recall inicial.

Maximización del F1-Macro mediante optimización (0.654): Al desplazar dinámicamente el umbral de decisión a 0.410, el Ensemble optimizado alcanza un F1-Macro de 0.654, superando el rendimiento de su propia versión base y el de cualquier otro clasificador individual de la comparativa. Esto demuestra estadísticamente que se logra el mejor balance global entre ambas condiciones.

Optimización del Recall de Intermedios (0.741): La reducción estratégica del threshold permite al modelo pasar de un Recall base de 0.524 (identificando 89 de los 170 jugadores Intermedios del conjunto de prueba) a un Recall de 0.741 (capturando correctamente 126 muestras). Esto representa una ganancia relativa del 41.6% en la sensibilidad de la clase minoritaria y desbalanceada, mitigando drásticamente el impacto de los falsos negativos.

Precisión de Intermedios bajo control normativo (0.500): A pesar de la importante ganancia en sensibilidad, la tasa de falsos positivos se mantiene bajo un estricto control técnico. La precisión específica para la clase minoritaria se sitúa en el 0.500, permaneciendo holgadamente por encima del umbral mínimo de tolerancia fijado en el alcance del proyecto (0.40). Esto asegura que una de cada dos alertas de nivel Intermedio es plenamente certera.

Compromiso asumible en la Exactitud Global (0.664): El coste computacional y operativo de priorizar la clase minoritaria se traduce en un sutil descenso del Accuracy global, pasando del 0.690 (Ensemble a threshold 0.5) a un 0.664. Esta desviación de apenas 2.6 puntos porcentuales es un precio menor e idóneo dado el beneficio crítico obtenido en la detección de la destreza del usuario.

Robustez frente al sobreajuste y estabilización probabilística: A diferencia de modelos como Random Forest (Accuracy de 0.676 pero con alta varianza interna) o SVM, el algoritmo de ensamble suaviza las fronteras de decisión. Al promediar de forma ponderada las salidas probabilísticas de modelos con naturalezas matemáticas tan dispares como la Regresión Logística y las Redes Neuronales, se mitigan los sesgos individuales, garantizando una excelente capacidad de generalización en entornos de producción.

En conclusión, el Ensemble con threshold ajustado a 0.410 representa el auténtico óptimo de Pareto para la memoria: maximiza la identificación de la clase crítica desbalanceada manteniendo una estabilidad predictiva global aceptable, todo ello bajo la restricción metodológica de operar con estadísticas agregadas de partidas, las cuales ofrecen una señal de habilidad competitiva valiosa pero no estrictamente determinante.

7. Manual de usuario

Para garantizar la total reproducibilidad de los experimentos descritos en esta memoria, el proyecto se ha estructurado como un pipeline de código robusto, parametrizable y autocontenido en Python. A continuación, se detallan los requisitos del entorno, la estructura del entregable y las instrucciones necesarias para su correcta explotación.

7.1 Requisitos del sistema y entorno

El desarrollo se ha diseñado para ser ejecutado de forma agnóstica al sistema operativo, siendo plenamente compatible con Windows, macOS y distribuciones Linux. Los requisitos mínimos para asegurar el funcionamiento del software son contar con una versión de Python 3.8 o superior, disponer del archivo de datos denominado `games.csv` ubicado obligatoriamente dentro de una carpeta llamada `data`, e instalar las dependencias de las librerías principales utilizadas, que corresponden a NumPy, Pandas, Scikit-Learn, Imbalanced-Learn para el algoritmo SMOTE, Matplotlib y Seaborn.

Para aislar estas librerías y evitar conflictos con otras configuraciones locales, se recomienda encarecidamente desplegar un entorno virtual limpio utilizando el módulo nativo `venv`. Para ello, el usuario simplemente debe abrir la terminal de su sistema operativo, acceder al directorio raíz del proyecto y ejecutar el comando de creación de entornos virtuales habitual de Python. Una vez generado, se debe proceder a su activación introduciendo en la línea de comandos la ruta hacia el script de activación, el cual se encuentra dentro de la carpeta de scripts binarios del propio entorno (utilizando el comando `source` seguido de la ruta correspondiente en sistemas basados en Linux o macOS, o ejecutando directamente el archivo ejecutable en entornos Windows). Finalmente, con el entorno virtual ya activo en la consola, se realiza la instalación automatizada de todas las dependencias requeridas ejecutando el comando de instalación de Pip apuntando de forma directa al archivo `requirements.txt` adjunto.

7.2 Estructura del código fuente

El código se organiza de manera modular siguiendo las buenas prácticas de la ingeniería de software para proyectos de aprendizaje automático:

Punto de entrada principal (`main.py`): Es el script coordinador encargado de secuenciar todo el pipeline del proyecto, incluyendo la lectura de datos, el preprocesado, el entrenamiento, la evaluación de métricas y la exportación de resultados.

Módulo de preprocesado (`preprocessing`): Contiene los scripts encargados de la limpieza inicial del dataset, la ingeniería de variables derivadas, la codificación de variables categóricas y el balanceo sintético mediante SMOTE.

Módulo de modelos (models): Almacena las clases y algoritmos de los clasificadores, destacando las implementaciones vectorizadas desde cero en NumPy para la Regresión Logística y la Red Neuronal, junto con los envoltorios optimizados de las librerías estándar.

Módulo de evaluación (evaluation): Agrupa las utilidades dedicadas al cálculo matricial de métricas como el F1-Macro, el ROC-AUC y el Recall, así como la validación cruzada y el guardado de gráficas.

Predictor de producción (production_predictor.py): Clase de abstracción diseñada para cargar el modelo recomendado y aplicar de forma transparente el umbral óptimo de decisión.

Scripts de validación (test_production.py y api_example.py): Pequeños archivos de prueba que simulan el consumo de predicciones para comprobar que el entorno funciona correctamente.

7.3 Instrucciones de ejecución

El script principal incorpora una interfaz de línea de comandos basada en argumentos para flexibilizar la ejecución según la capacidad de cómputo del equipo:

La ejecución estándar recomendada entrena el modelo óptimo de producción invocando el script principal `main.py` junto con la bandera `--skip-eda` para omitir el análisis exploratorio redundante y la bandera `--model rf` para centrar el cómputo exclusivamente en el algoritmo de Random Forest. Si se prefiere realizar una ejecución completa comparativa que evalúe absolutamente todo el pipeline, el usuario debe lanzar el script principal acompañado de la bandera `--model all`, lo que activará de forma secuencial el bloque de visualización gráfica, el entrenamiento de los cinco algoritmos de manera consecutiva y la generación de la matriz de decisión global (requiriendo un tiempo estimado de apenas 120 segundos en hardware convencional). Para entornos con serias limitaciones de hardware o memoria RAM, el script permite configurar el argumento `--n-samples` seguido del número de filas deseadas (por ejemplo, mil muestras) para limitar el tamaño del muestreo estratificado procesado, acelerando los tiempos y evitando desbordamientos de memoria del sistema.

7.4 Artefactos y resultados del pipeline

Al completarse el flujo de ejecución de manera exitosa, el sistema exporta de manera automatizada una serie de elementos esenciales para la verificación de los resultados:

En primer lugar, se crea una carpeta de gráficos (plots) donde se almacenan las distribuciones del análisis exploratorio, las matrices de confusión individuales y las curvas ROC combinadas. En segundo lugar, se genera una carpeta de modelos (models) que guarda los pesos y las instancias de los clasificadores en formatos binarios serializados para su reutilización inmediata. Por último, el sistema genera en la raíz del proyecto el archivo estructurado `config_produccion.json`, el cual

contiene los parámetros y thresholds optimizados de operación (como el umbral de 0.410 para el modelo Ensemble) requeridos por el entorno de explotación.

7.5 Simulación del despliegue en producción

Para validar la aplicabilidad real del modelo optimizado, el módulo del predictor de producción permite instanciar el clasificador final directamente desde sistemas externos, simulando la integración en microservicios o aplicaciones web de toma de decisiones en tiempo real.

El flujo lógico del código en un entorno de producción sigue una secuencia sencilla: el sistema importa el módulo del predictor y lo instancia seleccionando el algoritmo recomendado; de manera interna, el software lee de forma automática el archivo de configuración en formato JSON para extraer el umbral optimizado que equilibra la precisión y el recall.

Finalmente, al recibir un diccionario de datos con las diecisiete variables procesadas de una partida de ajedrez nueva (el cual puede ser testeado ejecutando de manera directa el script de validación `test_production.py`), el predictor aplica las transformaciones correspondientes cargando el archivo binario del modelo desde la carpeta nativa mediante la librería `pickle`, consulta el umbral mapeado en el archivo de configuración y devuelve un resultado estructurado con la clase predicha, clasificando con éxito si el perfil del jugador corresponde a un nivel Principiante o Intermedio.

8. Conclusiones

8.1 Resumen de resultados

Este trabajo ha abordado el problema de clasificar el nivel de habilidad de jugadores de ajedrez en dos categorías (Principiante e Intermedio) a partir de estadísticas observables de sus partidas, sin hacer uso directo de la puntuación ELO como variable de entrada. Para ello se han implementado y evaluado cuatro modelos de clasificación, Regresión Logística One-vs-All, Red Neuronal Feedforward, Support Vector Machine con kernel RBF y Random Forest, así como un Ensemble de votación suave que combina sus predicciones ponderadas por ROC-AUC.

El mejor resultado global se obtiene con el Ensemble con threshold ajustado a 0, que alcanza un F1 macro de 0.653 y un recall de Intermedios del 74 %, detectando correctamente 126 de los 170 jugadores de nivel intermedio presentes en el conjunto de prueba. Ningún modelo supera el 70 % de accuracy, lo que refleja la dificultad intrínseca del problema: las estadísticas agregadas de una partida contienen información limitada sobre el nivel del jugador cuando no se tiene acceso a la calidad de las jugadas individuales.

Entre los modelos individuales, la Regresión Logística OvA ofrece el mejor equilibrio entre clases a threshold por defecto (0.5), con un recall de Intermedios del 63 % y el mayor F1 macro individual (0.644). La Red Neuronal, pese a liderar en accuracy (0.700) y ROC-AUC (0.737), presenta el peor recall de Intermedios (0.43), lo que evidencia que una mayor capacidad expresiva del modelo no se traduce necesariamente en mejor rendimiento cuando la señal disponible en los datos es limitada y el desbalance de clases no se compensa adecuadamente en la frontera de decisión. La SVM es el modelo con rendimiento global más bajo en este problema, mientras que el Random Forest muestra un sobreajuste pronunciado que limita su generalización hacia la clase minoritaria.

El ajuste de threshold ha demostrado ser una técnica sencilla y efectiva para mejorar el recall de la clase minoritaria sin modificar el modelo entrenado. La búsqueda sistemática del umbral óptimo ha permitido incrementar el recall de Intermedios entre 14 y 42 puntos porcentuales según el modelo, a un coste moderado en precisión y accuracy. Este resultado subraya la importancia de no asumir que el threshold por defecto de 0.5 es el más adecuado en problemas con desbalance de clases.

8.2 Limitaciones

A pesar de los resultados obtenidos, el trabajo presenta una serie de limitaciones que conviene señalar con el fin de contextualizar adecuadamente su alcance:

En primer lugar, las features disponibles corresponden a estadísticas agregadas de la partida (número de movimientos, tipo de victoria, apertura, control de tiempo), que capturan el comportamiento global pero no la calidad de las decisiones individuales. Variables como el número de errores por movimiento, la evaluación de posiciones clave por parte de un motor de ajedrez o el

porcentaje de jugadas óptimas serían indicadores directos del nivel de habilidad que el conjunto de features actual no puede replicar.

En segundo lugar, el problema de clasificación se construye a partir de un umbral de discretización del ELO basado en percentiles, lo que introduce una cierta arbitrariedad en la definición de las clases. La zona de solapamiento entre Principiante e Intermedio, visible en el histograma de la Figura 1, implica que existen partidas genuinamente ambiguas cuya etiqueta correcta es discutible, lo que establece un techo de rendimiento difícil de superar con las features disponibles.

En tercer lugar, el tamaño del conjunto de datos utilizado (2.500 partidas muestreadas) es moderado. Las curvas de aprendizaje muestran que tanto la SVM como el Random Forest no han alcanzado su techo de rendimiento con los datos disponibles, lo que sugiere que ampliar el dataset podría traducirse en mejoras reales de generalización.

Finalmente, el desbalance de clases (66 % / 34 %) persiste en los conjuntos de validación y test, lo que limita estructuralmente el recall de la clase minoritaria incluso después de aplicar SMOTE en entrenamiento y ajustar el threshold de decisión.

8.3 Posibles mejoras

A partir de las limitaciones identificadas, se proponen las siguientes líneas de mejora para trabajos futuros:

La mejora más impactante sería enriquecer el conjunto de features con información derivada de las jugadas individuales. El acceso a la secuencia de movimientos de cada partida permitiría calcular métricas como el porcentaje de jugadas óptimas según un motor de evaluación, el número de errores graves o el tiempo por jugada, que son indicadores directos de la habilidad del jugador y reducirían significativamente el solapamiento entre clases.

Una segunda línea de mejora consistiría en utilizar el dataset completo de Lichess, con más de 20.000 partidas disponibles, en lugar de la muestra de 2.500 empleada en este trabajo. Las curvas de aprendizaje indican que los modelos siguen mejorando con más datos, por lo que este cambio podría traducirse en ganancias de rendimiento sin necesidad de modificar la arquitectura de los modelos.

Desde el punto de vista de los algoritmos, sería interesante explorar modelos de gradient boosting como XGBoost o LightGBM, que suelen obtener resultados superiores a Random Forest en problemas de clasificación con datos tabulares y desbalance de clases, gracias a su capacidad de construir árboles de forma secuencial corrigiendo los errores del modelo anterior.

Por último, el ajuste de threshold podría refinarse mediante el análisis de la curva de precisión-recall completa en lugar de una búsqueda por pasos, lo que permitiría identificar con mayor precisión el punto de operación óptimo según los requisitos específicos de la aplicación: si se prioriza maximizar la detección de Intermedios o mantener una precisión mínima garantizada.

9. Bibliografía

- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
- Chawla, N. V., Bowyer, K. W., Hall, L. O., y Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
<https://doi.org/10.1613/jair.953>
- Cortes, C., y Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.
<https://doi.org/10.1007/BF00994018>
- Datasnaek. (2017). Lichess Games Dataset [Conjunto de datos]. Kaggle.
<https://www.kaggle.com/datasets/datasnaek/chess>
- Goodfellow, I., Bengio, Y., y Courville, A. (2016). *Deep Learning*. MIT Press.
<https://www.deeplearningbook.org/>
- He, K., Zhang, X., Ren, S., y Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. En *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* (pp. 1026–1034). <https://doi.org/10.1109/ICCV.2015.123>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., y Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://www.jmlr.org/papers/v12/pedregosa11a.html>